实验报告

作业十二—定点 IIR 滤波器

姓名：　　王峤宇

学号：　　2025180010270 21

目　　录

1　　实验前言 1
2　　实验内容 2
3　　实验结果 6
4　　实验总结 10

1 实验前言

1.1 实验目的

（1）利用 Matlab 设计 120Hz 四阶 IIR 高通滤波器（采样率 8000Hz）。

（2）用 C 语言实现定点 IIR 滤波器，分别实现 16-bit 和 32-bit 的定点滤波器，对输入语音信号进行滤波。

（3）比较 Matlab（float64）滤波与 C 语言定点实现的滤波结果，分析结果。

1.2 实验方法

经过调研，Matlab 的滤波器实现默认采用 float64 数据类型进行，而 Python 的 scipy 的 signal 包同样是默认采用 float64 进行滤波操作，使用 python 替代 matlab 进行实验操作。

进一步，对 Matlab 的 filterdesigner 工具的定点系数导出进行调研，确定其采用的定点策略是，根据滤波器的系数范围自动确定定点的 Q 值，本次实验过程中，采用 python 设计完成滤波器系数之后，自动确定 Q 值，输出对应量化好的 16bit 和 32bit 的系数值，供 C 语言使用。

2 实验内容

2.1 滤波器设计

2.1.1 python 端滤波器设计

首先使用 python 设计滤波器（scipy.signal.butter），滤波器设计过程中使用的参数如表 2-1 所示，C 语言中使用二阶节结构实现，python 的 butter 滤波器设计函数可以直接导出 SOS 结构的系数。

表 2-1 滤波器设计参数

参数	值
滤波器类型	巴特沃斯高通滤波器
阶数	4
截止频率	120Hz
实现结构	二阶节级联，2 个 biquad

4 阶滤波器由两个二阶节实现，对应的系统函数如式（2.1），拆分后的二阶节标准写法为式（2.2），a_0系数被整理为 1，在后续的 C 语言定点滤波器系数设计中省略该值，相对应的每一个二阶节需要导出的滤波器系数为 5 个，4 阶滤波器共 10 个滤波器系数。

$$H(z) = H_1(z) \cdot H_2(z) \tag{2.1}$$

$$H_k(z) = \frac{b_{k0} + b_{k1}z^{-1} + b_{k2}z^{-2}}{1 + a_{k1}z^{-1} + a_{k2}z^{-2}} \tag{2.1}$$

实际设计得到的滤波器系数如表 2-2 和表 2-3 所示，滤波器的系数本身的范围在$[-2, 1]$之间，无论是 16-bit 还是 32-bit 的量化方案，都默认采用了一位整数，一位符号数的方案，刚好覆盖了系数范围。

表 2-2 第一个 Biquad 系数

数据类型	Q	a_1	a_2	b_0	b_1	b_2
Float64		-1.8318	0.8400	0.8840	-1.7681	0.8840
16-bit	14	-30013	13763	14485	-28970	14485
32-bit	30	-1966938108	901964539	949286603	-1898573205	949286603

表 2-3 第二个 Biquad 系数

数据类型	Q	a_1	a_2	b_0	b_1	b_2
Float64		-1.9219	0.9304	1.0000	-2.0000	1.0000
16-bit	14	-31489	15245	16384	-32768	16384
32-bit	30	-2063633961	999091444	1073741824	-2147483648	1073741824

由于直接采用符号数量化，可以覆盖到-2（第二个二阶节的b_1）这个系数下界，

Q14 的定点量化表示范围对应$[-2, 2-2^{-14}]$，Q30 的定点量化表示范围为$[-2, 2-2^{-30}]$，保证覆盖系数。

2.1.2 C 语言端滤波器设计

整体滤波器设计采用 linux 风格的 C 语言面向对象设计，相关的结构体如图 2-1 所示，额外提供 init 方法进行初始化，process 处理单个样本，以及 process signal 对信号进行滤波处理。其中 BiquadCoeffs16 存储定点化的二阶节系数，顶点数据对应的 Q 值保存在顶层顶层描述符 IIRFIlter16 中；BiquadState16 为每个二阶节提供延迟单元对应的存储资源，其中存储延迟结果的位宽也限制为 16-bit，中间累加结果通过饱和计算限制为 16-bit。

```c
/* ── 16-bit 类型 ──────────────────────────────────────────────────────────── */

/** @brief 单个双二阶节系数（Q 格式，a0=1 隐含） */
typedef struct {
    int16_t b0, b1, b2;     /**< 前馈系数（分子） */
    int16_t a1, a2;         /**< 反馈系数（分母） */
} BiquadCoeffs16;

/** @brief 单个双二阶节状态变量（直接II型转置） */
typedef struct {
    int32_t s1;             /**< 第一个延迟单元 */
    int32_t s2;             /**< 第二个延迟单元 */
} BiquadState16;

/** @brief 16-bit 定点 IIR 滤波器描述符 */
typedef struct {
    BiquadCoeffs16 sections[IIR16_MAX_SECTIONS];  /**< 各二阶节系数 */
    BiquadState16  states[IIR16_MAX_SECTIONS];    /**< 各二阶节状态 */
    int16_t num_sections;   /**< 实际二阶节数量 */
    int16_t Q;              /**< 系数量化小数位数 */
    int64_t clamp_min;      /**< 饱和下限 */
    int64_t clamp_max;      /**< 饱和上限 */
} IIRFilter16;
```

图 2-1 16-bit 定点滤波器描述符

最后，定点信号处理系统最重要的点在于计算饱和的保证，在顶层描述符中提供饱和计算的上下限，以方便不同位宽顶点滤波器使用相同的描述符接口。在初始化的过程中，16-bit 定点滤波器的饱和上下限默认设定为 INT16 的范围，如图 2-2 所示，SOS 结构方便用标准的数组结构初始化和实现一个 IIR 滤波器。

具体的滤波通过单个样本的步进计算接口实现，如图 2-3 所示，对信号的滤波处理直接 for 循环调用 process 接口实现。简化设计，验证定点原理，在计算过程中使用的 acc 和输出缓存的中间值采用 int64 保存，然后用 rshift_round 将结果右移

Q 位并四舍五入，将系数 Q 值对计算结果的影响抵消，这样的话输出数据的 Q 永远与输入数据的 Q 值保持一致，将定点滤波器的 Q 值影响隐藏，最后通过 sat 函数将计算结果限制在饱和范围（INT_16_MIN 到 INT_16_MAX）内重新保存为 16-bit 的值。

```
26  void iir_filter_16_init(IIRFilter16 *f, const int16_t *coeffs_flat,
27                          int num_sections, int Q) {
28      memset(f, 0, sizeof(*f));
29      f->num_sections = (int16_t)num_sections;
30      f->Q            = (int16_t)Q;
31      f->clamp_min    = INT16_MIN;
32      f->clamp_max    = INT16_MAX;
33
34      for (int s = 0; s < num_sections; s++) {
35          const int16_t *c = coeffs_flat + s * 5;
36          f->sections[s].b0 = c[0];
37          f->sections[s].b1 = c[1];
38          f->sections[s].b2 = c[2];
39          f->sections[s].a1 = c[3];
40          f->sections[s].a2 = c[4];
41      }
42  }
43
```

图 2-2 饱和范围初始化

```
44  // 输出值与输入的Q格式相同
45  int16_t iir_filter_16_process(IIRFilter16 *f, int16_t x) {
46      int64_t xn = (int64_t)x;
47      int Q = f->Q;
48      int64_t lo = f->clamp_min, hi = f->clamp_max;
49
50      for (int s = 0; s < f->num_sections; s++) {
51          BiquadCoeffs16 *c = &f->sections[s];
52          BiquadState16  *st = &f->states[s];
53
54          /* y[n] = b0*x[n] + s1 */
55          int64_t acc = (int64_t)c->b0 * xn + ((int64_t)st->s1 << Q);
56          int64_t yn = sat(rshift_round(acc, Q), lo, hi);
57          // Q14乘完之后，是Qin + 14，右移丢弃14个小数，并限制饱和值，结果是Qin
58
59          /* s1_next = b1*x[n] - a1*y[n] + s2 */
60          acc = (int64_t)c->b1 * xn - (int64_t)c->a1 * yn + ((int64_t)st->s2 << Q);
61          st->s1 = (int16_t)sat(rshift_round(acc, Q), lo, hi);
62
63          /* s2_next = b2*x[n] - a2*y[n] */
64          acc = (int64_t)c->b2 * xn - (int64_t)c->a2 * yn;
65          st->s2 = (int16_t)sat(rshift_round(acc, Q), lo, hi);
66
67          xn = yn;
68      }
69      return (int16_t)xn;
70  }
```

图 2-3 二阶节遍历计算单个样本

虽然中间计算转换到了 64bit 实现，但是 64bit 的转化只是方便定点计算的实

现，所有最后的结果、中间值、缓存值都通过 sat 饱和（图 2-4）限制在了 16-bit，本质对应定点计算系统，最终直接用软件实现了对应的定点硬件系统过程。

综上，32bit 的定点滤波器仅仅是修改了 16bit 定点滤波器的数据类型转化为 int32，并将饱和值限制在 INT_32 的范围。

```
/** @brief 带四舍五入的右移 */
static int64_t rshift_round(int64_t val, int Q) {
    return (val + ((int64_t)1 << (Q - 1))) >> Q;
}

/** @brief 饱和限幅 */
static int64_t sat(int64_t val, int64_t lo, int64_t hi) {
    return val < lo ? lo : (val > hi ? hi : val);
}
```

图 2-4 定点工具

2.1.3 数据预处理

采用 WAV 的格式的语音信号，首先对数据下采样到 8kHz，然后将数据保存为 Q15 和 Q31 两种量化格式，提供给 C 语言定点滤波器使用。

```
# ── 5b. 量化输入数据并保存 ──────────────────────────────────────
print(f"[5b] Quantizing input to Q{coeff_q16_bits} / Q{coeff_q32_bits}…")
x_norm = x_8k / DATA_SCALE  # float ∈ [-1, 1)

# 16-bit: Q{coeff_q16_bits} → int16_t raw
q16_int = np.round(x_norm * q16_scale).astype(np.int16)
q16_int.tofile(os.path.join(DATA_DIR, 'input_16bit.raw'))

# 32-bit: Q{coeff_q32_bits} → int32_t raw
q32_int = np.round(x_norm * q32_scale).astype(np.int32)
q32_int.tofile(os.path.join(DATA_DIR, 'input_32bit.raw'))
```

图 2-5 design_filter 对输入数据的量化(Q15 和 Q31)

3 实验结果

在实验结果的分析，直接比对定点滤波返回结果的.wav 音频难以分辨出差距，也可以得到结论，即定点处理系统对语音信号的影响很小。但为了实验结果的可视化与理论分析，从量化误差、SNR 的角度对实验结果进行分析，以 python 的 float64 运算结果为参考信号，对滤波器这个系统量化的影响进行分析。

3.1 实验流程

本次实验共 5 个文件，如下：

（1）design_filter.py：使用 python 设计滤波器，量化滤波器系数和语音信号。

（2）main.c：读取系数和输入、初始化滤波器、完成滤波、输出结果。

（3）iir_filter.c、iir_filter.h：定点 IIR 滤波器算法实现。

（4）analyze_results.py：分析实验结果，计算 SNR。

考虑到 Linux 和 windows 的实验环境兼容，本实验提供 main.py 作为自动化脚本运行实验，可使用 python main.py 执行，在 linux 环境也提供相应的 Makefile。

3.2 实验结果分析

3.2.1 时域对比

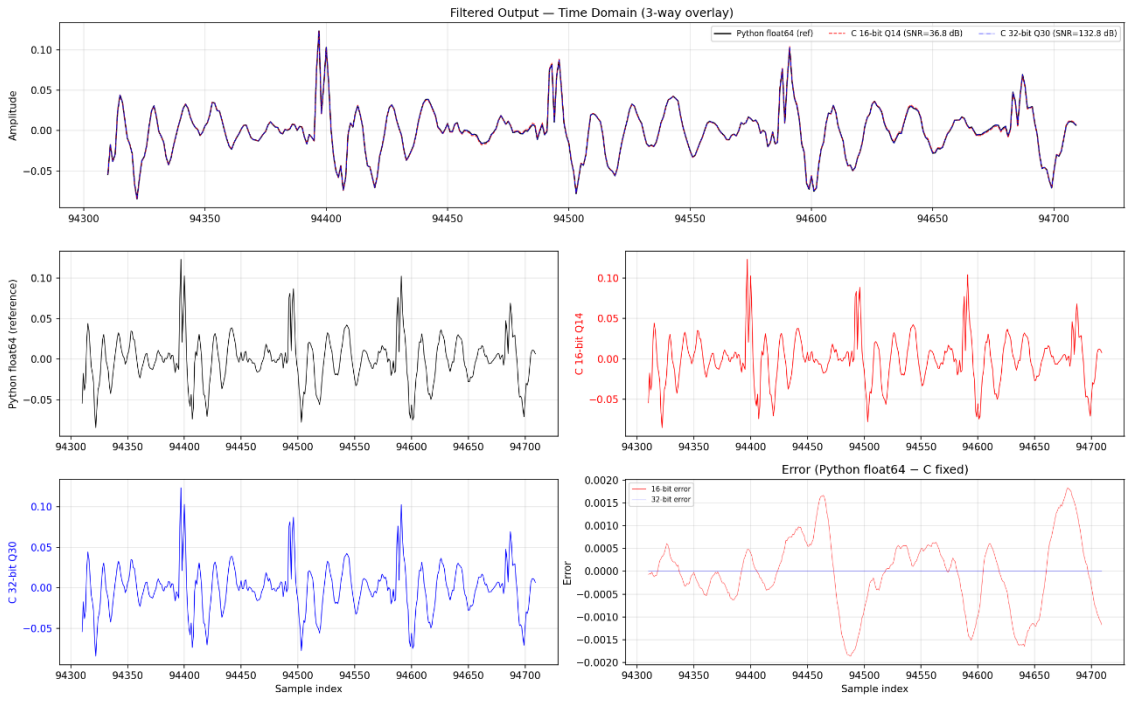


图 3-1 时域结果对比

节选部分片段的时域结果对比如图 3-1 所示，最后一张图绘制的是以 float64 为参考的 16-bit 和 32-bit 的误差结果对比。肉眼上很难观察到三种滤波器系统的滤波结果有何差异，最上面的图对应了三种结果叠在一起，32-bit（蓝色）与 float64 参考（黑）几乎无法区分，看不到偏移的蓝色，仅 16-bit（红色）可以看到轻微的偏差，包括实际听滤波结果的.wav，也难以发现明显的差距。但误差的关系十分明显，首先 32-bit 的滤波器误差很小，基本接近 0，但是 16-bit 的结果在这段峰值接近 0.002，而且误差好像呈现了一定的规律，输出信号幅值接近 0 的地方误差较大，对此进行进一步分析。

全部采样点的误差信号全貌如图所示，在标题中包含了对应的 MSE 和 SNR 计算结果，32-bit 基本不受影响，SNR 高达 132.8dB，MSE 数量级-16。而 16-bit 相比 32-bit 仅少了 16-bit 的资源开销，其 SNR 仅 36.8dB。



图 3-2 误差信号全貌

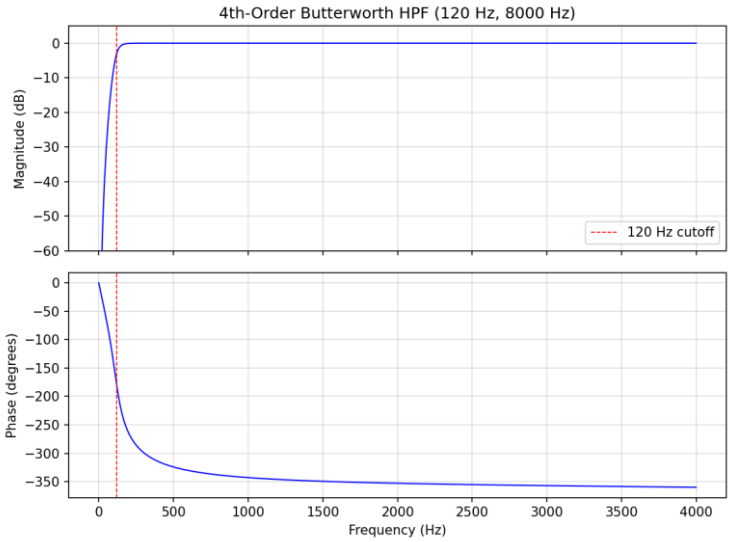


图 3-3 滤波器频率响应

针对 16-bit 结果误差在不同区域幅值不同呈现一定的规律性，从误差的功率谱进行分析，首先滤波器的频率响应如图 3-3 所示，滤波器输出信号的功率谱如图 3-4 所示。全频带图中可以看到，三路频谱高度重合，仅在低频过渡带附近可观察到 16-bit 的微小偏差。

低频放大图（0–500Hz）清晰展示了量化精度对滤波器频率响应的影响—16-bit Q14 的系数量化系统，在阻带（<120Hz）出现约 3–5dB 的频谱偏差，其原因本质是系数量化截断导致了足够影响结果的零极点偏移，原本滤波器系统设计好的高通滤波器的阻带性能受到影响。

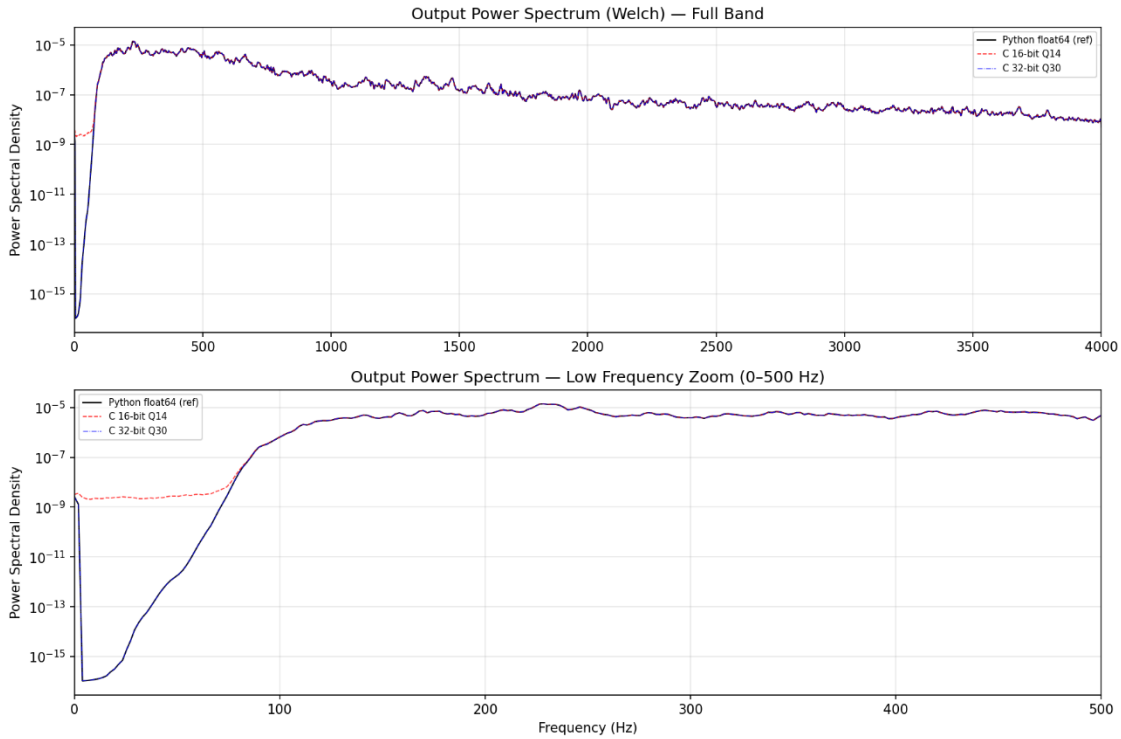


图 3-4 输出功率谱

误差的功率谱如图 3-5 所示。在功率谱和时域图中观察不到 32-bit 定点量化带来的影响，在误差的功率谱中可以明显观察到其误差的功率谱实际上与 16-bit 的误差功率谱的形状完全一致，其形状与系统的频率响应（图 3-3）呈现一种对偶的关系，随着频率的提高，误差的功率谱密度是逐渐接近量化误差理论功率谱密度的值（频率越高，频响越接近 1），理论功率谱密度采用量化误差白噪声模型计算，量化噪声功率如下所示，其中Δ对应实际的量化步长。

$$P_q = \frac{\Delta^2}{12}$$

Q14 的实际量化噪声功率谱密度是：

$$S_e(f) = \frac{P_q(\Delta = 2^{-14})}{8000} = 3.88 \times 10^{-14}$$

Welch 功率谱密度对实信号返回单边谱，对应结果:

$$S_1(f) = 2S_e(f) = 7.76 \times 10^{-14} = -131.1\text{dB/Hz}$$

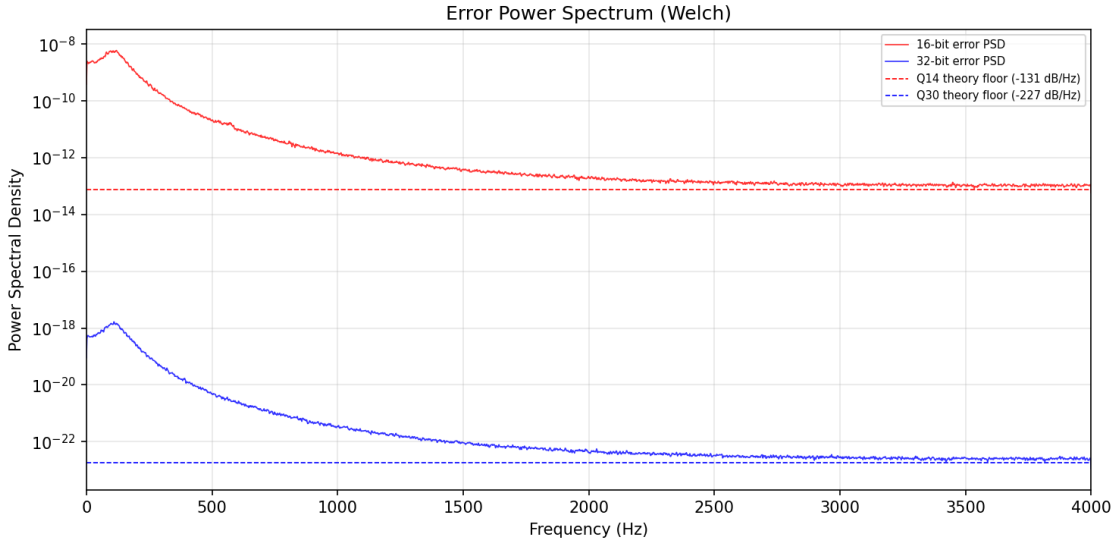


图 3-5 误差功率谱

综上，在这个定点系统中，其输出误差的噪声大于由量化引起的理论误差，所以一定由其他的噪声引入（误差）。即上文分析的系统模型本身零极点偏移带来的系统误差，IIR 滤波器最大的特点是需要使用极点，极点对系统稳定性的影响较大，而这种影响可以视为对原本高通滤波器性能的削弱，所以其功率谱噪声体现为频率响应的互补形式低通形状。

4 实验总结

本科与研究生方向都以硬件为主，对该实验非常感兴趣，很庆幸得到了一些有意思的结论。定点算法第一印象的理解就是通信原理中，对信源编码分析采用的量化噪声模型，实际迁移到滤波器系统的分析中，可以采用对应的扰动模型，即零极点扰动模型，而各个扰动随机变量是量化带来的影响，其方差对应的白噪声的功率谱密度，剩余的就是滤波器系统本身参数的影响，使用泰勒展开得到的扰动模型近似结果如下，偏导对应系数的变化灵敏度。

$$S_e(\omega) = S_x(\omega)\,\frac{\Delta^2}{12}\sum_{i=1}^{N}\left|\frac{\partial H\left(e^{j\omega}\right)}{\partial c_i}\right|^2$$

最后是一些遗憾，直接从功率谱分析的结果可以得到误差中包含最初量化引入的误差以及系统零极点偏移引入的系统误差，但没有考虑计算过程中中间值饱和操作带来的误差，这部分难以观察与进行理论建模，能想到的方法是，将中间计算过程中饱和丢弃的值保存下来，最后分析其对输出的影响。